System Architecture Document

Agentic SDLC Automation Platform

React + Django + LangChain + MCP

Version: 1.0

Status: Draft for Architecture Review

Date: March 22, 2026

Intended Audience: Engineering, Platform, DevOps, Security, Enterprise Architecture

1. Purpose and Scope

This document defines the target system architecture for the Agentic SDLC Automation Platform. The platform is a governed, enterprise web application that enables software teams to execute and design agent-driven software delivery workflows across Jira, GitHub, code generation, pull request preparation, review, and deployment to development environments.

The architecture uses React for the primary interactive user experience, Django and Django REST Framework for backend APIs and orchestration services, LangChain for agentic workflow orchestration, an MCP server as the enterprise tool execution layer, and H2 as the initial relational database with a design that supports later migration to PostgreSQL, MySQL, SQL Server, or Oracle.

This document covers logical architecture, component architecture, integration design, workflow orchestration, data architecture, security architecture, deployment architecture, and operational considerations.

2. Architectural Goals

The architecture is designed to achieve the following goals:

1. Provide a governed, human-in-the-loop SDLC automation platform.
2. Separate user experience, workflow orchestration, agent reasoning, tool execution, and governance responsibilities.
3. Reuse the existing MCP server for tools, prompts, and resources rather than rebuilding connectors.
4. Support interactive, team-defined agentic workflow creation through a visual flow builder.
5. Ensure that all high-impact actions are transparent, policy-controlled, auditable, and attributable.
6. Support phased adoption starting with development-environment automation only.
7. Allow future database portability and enterprise scaling without major architectural rewrite.

3. Guiding Principles

The following principles govern the design:

- Workflow state must be explicit and durable.
- Agent behavior must be bounded by approved tools, prompts, and policies.
- MCP is the execution layer, not the governance layer.
- High-risk actions must pass through approval checkpoints.
- Tool invocation must be mediated by the backend.
- Flow definitions must be versioned and reusable.
- Every major action must be observable through logs, metrics, and audit records.

4. Technology Stack

Presentation Layer

• React for end-user application, flow builder, workflow dashboards, diff review, and execution monitoring.

• Django Admin for administrative functions such as capability registry, flow governance, role policy, and configuration.

Application Layer

• Django for core web framework.

• Django REST Framework for API services.

• Python service layer for workflow engine, state transitions, policy enforcement, and MCP interaction.

• LangChain for agentic workflow orchestration, prompt chains, tool-routing abstractions, structured outputs, and multi-step agent execution.

Data Layer

• H2 relational database for initial implementation, with strict portability requirements so that it can be replaced later without domain model changes.

• Optional object storage in later phases for large artifacts, execution logs, and exported session data.

Integration Layer

• MCP client implemented in Python for tool, prompt, and resource discovery and invocation.

• MCP server as the enterprise integration layer for Jira, GitHub, CI/CD, repository metadata, deployment, and other tools.

Execution and Operations

• Celery or RQ for background task processing.

• Enterprise SSO for authentication.

• Reverse proxy / load balancer for ingress.

• Containerized deployment for backend, frontend, and workers.

5. High-Level Logical Architecture

The platform is organized into six primary layers:

1. User Interaction Layer
   React web application for developers, reviewers, team leads, and administrators.

2. Application and API Layer
   Django services exposing workflow APIs, flow-builder APIs, session APIs, review APIs, and administrative APIs.

3. Orchestration and Agent Layer

LangChain-based orchestration and agent runtime executing bounded workflow tasks using approved prompts, tools, and context.

4. Governance and Policy Layer

Role-based access control, tool authorization, approval gating, branch protections, environment restrictions, and audit policy enforcement.

5. Integration Layer

MCP client and capability registry used to discover and invoke MCP tools, prompts, and resources.

6. Data and Persistence Layer

H2 database used for users, sessions, flow templates, execution history, approvals, PR records, deployments, and audit logs.

6. Core Components

6.1 React Frontend

The React application is the primary user interface. It provides the dashboard, story intake screens, implementation planning workspace, diff review screens, pull request preparation screens, deployment console, audit timeline, and interactive flow builder.

6.2 Django API Gateway

Django REST APIs serve all frontend requests, validate user identity, enforce permissions, normalize data models, and broker all backend operations.

6.3 Workflow Engine

The workflow engine manages session creation, workflow state transitions, execution planning, retry policies, compensation behavior, and orchestration checkpoints.

6.4 LangChain Orchestrator

LangChain is used to implement bounded agents, chains, tool-aware workflows, structured outputs, prompt templates, and step-wise execution logic. LangChain operates inside application guardrails and must not bypass policy controls.

6.5 MCP Client Service

This service discovers available MCP tools, prompts, and resources, invokes them, validates payloads, normalizes responses, and records execution traces.

6.6 Policy and Approval Service

This component enforces role-based tool access, approval requirements, branch restrictions, environment restrictions, and publication controls for custom workflows.

6.7 Session and Audit Service
Stores workflow sessions, execution history, approvals, agent outputs, tool-call metadata, and compliance-relevant activity traces.

6.8 Background Worker Service
Runs long-running or asynchronous tasks such as agent execution, repository scans, PR drafting, and deployment status polling.

7. Detailed Component Interactions

Primary request flow:
1. User interacts with React UI.
2. React calls Django REST API.
3. Django authenticates and authorizes the request.
4. Workflow Engine determines current session state and allowed action.
5. LangChain orchestrator runs the appropriate bounded chain or agent.
6. If external actions or context are required, MCP client invokes the approved MCP tool or resource.
7. Responses are normalized by backend services.
8. Session state, outputs, approvals, and audit logs are persisted.
9. React receives structured results for display.

This model ensures that neither the frontend nor LangChain directly performs high-risk external operations without backend mediation.

8. LangChain Architecture

LangChain is retained as the agentic workflow orchestration framework.

8.1 Role of LangChain
• Execute bounded agents for story analysis, planning, code proposal, test generation, PR packaging, review readiness, and deployment readiness.
• Manage structured prompt chains.
• Support tool-aware reasoning while remaining confined to the policy-approved MCP tool set.
• Produce typed outputs that can be rendered directly in the UI and stored in the database.

8.2 LangChain Design Constraints
• LangChain must not be allowed to call arbitrary tools.
• Allowed tools must be injected per workflow step and per user role.
• All agent outputs must be structured and persisted.
• Human approval is required before any high-impact action continues to the next state.

8.3 Recommended LangChain Patterns
• PromptTemplate / ChatPromptTemplate for standardized prompt construction.

• Runnable sequences for deterministic multi-step planning.
• Tool-enabled agents for bounded tool invocation.
• Structured output parsers / Pydantic schemas for consistent outputs.
• Memory limited to workflow-session context rather than unconstrained conversational memory.

9. MCP Integration Architecture

The platform treats MCP as the enterprise execution plane.

9.1 MCP Usage Categories
• Tools: create branch, fetch story, create PR, trigger deployment, read repo metadata, fetch code context.
• Resources: coding standards, architecture docs, repo metadata, templates, policies, design references.
• Prompts: standardized enterprise prompt templates for story decomposition, PR summaries, review checklists, and deployment readiness checks.

9.2 Capability Registry
At startup and on schedule, the backend discovers available MCP capabilities and maps them to internal workflow functions. Capabilities are classified as read-only, write-capable, approval-required, or environment-sensitive.

9.3 Execution Rules
• All MCP invocations must be initiated by the backend only.
• Tool arguments must be validated server-side.
• Unknown write-capable tools must be disabled by default.
• Every MCP call must generate an audit record and execution trace.

10. Interactive Flow Builder Architecture

The platform includes a self-service flow builder for authorized users.

10.1 Flow Builder Frontend
The React flow builder provides a canvas-based drag-and-drop experience using nodes and edges. Users can assemble triggers, agent steps, MCP tool steps, resource fetches, approvals, condition branches, notifications, PR actions, and deployment actions.

10.2 Flow Definition Model
Flow definitions are stored as versioned JSON documents with:
• metadata
• node list
• edge list
• node configuration

• role restrictions
• publishing state
• validation status

10.3 Backend Flow Validation
Django validates flow definitions for:
• missing required inputs
• unauthorized tool usage
• invalid transitions
• missing approvals for risky actions
• unreachable nodes
• loops without a valid termination path

10.4 Runtime Execution
When a flow is executed, the engine converts the stored definition into a runtime execution plan. LangChain handles agent tasks; MCP handles external tools; the workflow engine manages sequence, conditions, approvals, and persistence.

11. Workflow State Machine

Each workflow run is persisted as a stateful session.

Minimum core states:
• Session Created
• Story Selected
• Story Analyzed
• Plan Generated
• Plan Approved
• Repository Confirmed
• Branch Created
• Context Prepared
• Change Proposal Generated
• Changes Reviewed
• PR Draft Created
• Review Pending
• Review Approved
• Deployment Pending
• Deployed to Development
• Rework Required
• Closed

State transitions are enforced by the workflow engine. Invalid transitions are blocked. High-risk actions are only available in permitted states.

12. Data Architecture

The initial implementation uses H2, but the data model must remain database-portable.

12.1 Core Entities

- User
- Role
- Team
- WorkflowTemplate
- WorkflowTemplateVersion
- WorkflowRun
- WorkflowStepRun
- AgentRun
- MCPToolCall
- ApprovalRecord
- PullRequestRecord
- DeploymentRecord
- AuditEvent

12.2 Persistence Rules

- Flow definitions are versioned.
- Workflow runs are immutable in historical state but may contain updated current-state fields.
- Agent outputs are stored with schema version information.
- Tool inputs and outputs are stored with redaction rules where needed.
- Audit events are append-only.

12.3 Database Portability

The application should use ORM-managed abstractions and avoid H2-specific SQL constructs, allowing later migration to enterprise-grade RDBMS platforms.

13. API Architecture

The API layer is implemented with Django REST Framework.

Primary API groups:

- Authentication / session APIs
- Jira story intake APIs
- Workflow template APIs
- Workflow run APIs
- Agent execution APIs
- Diff review APIs
- PR APIs
- Approval APIs
- Deployment APIs

• Audit / reporting APIs
• Capability registry APIs

API design principles:
• resource-oriented endpoints
• role-aware authorization
• idempotency for sensitive actions where possible
• structured error responses
• correlation IDs for observability

14. Security Architecture

Security is a first-class architectural concern.

14.1 Identity and Access
• Enterprise SSO is required.
• RBAC governs access to repos, flows, actions, and admin features.
• Sensitive actions may require step-up confirmation depending on policy.

14.2 Backend Mediation
• The browser must never hold privileged credentials for Jira, GitHub, deployment, or MCP write actions.
• All external tool execution occurs through the backend.

14.3 Secret Handling
• Secrets must be stored in a secure secret-management mechanism and never stored in frontend state.
• Prompt and log outputs must support redaction.

14.4 Prompt Injection and Content Safety
• Story text, repo content, comments, and documents must be treated as untrusted input.
• Context passed to LangChain must be scoped, filtered, and policy-checked.
• Tool execution must never rely on model output without validation.

14.5 Audit and Non-Repudiation
• High-impact actions must be logged with identity, timestamp, action, target, and result.

15. Deployment Architecture

Recommended deployment model:

• React frontend deployed as static assets behind enterprise ingress.
• Django API deployed as containerized services.
• Worker containers for background tasks.

• H2 used only for initial environments or lower-scale deployment; migration path to production-grade database should be planned early.
• Reverse proxy / ingress controller terminates TLS.
• MCP connectivity is restricted to backend network paths.
• CI/CD pipelines build and deploy frontend and backend components independently.

Environments:
• Local developer environment
• Shared development environment
• QA / test environment in later phases
• Production environment for platform itself (distinct from app deployment targets)

The platform initially supports deployment only to development target environments for user code workflows.

16. Observability and Operations

Operational observability must include:

• structured application logs
• workflow execution logs
• MCP invocation traces
• LangChain step metrics
• approval wait-time metrics
• deployment success and failure metrics
• dashboard for stuck sessions and failed steps

Recommended operational telemetry:
• correlation ID per request
• session ID per workflow run
• step ID per execution step
• tool-call ID per MCP invocation

17. Scalability and Performance Considerations

The architecture should support incremental scaling.

• React frontend scales independently from backend services.
• Django API and worker services can scale horizontally.
• Long-running agent operations should be offloaded to workers.
• Session and audit queries should be indexed for timeline and reporting access.
• Flow execution should be resumable after transient failure.
• H2 is acceptable for initial use, but not a long-term scalability target for enterprise scale.

18. Failure Handling and Recovery

Failure handling strategy:

- retry transient MCP and network failures
- persist partial execution state before and after each step
- mark failed workflow steps explicitly
- allow authorized users to resume, retry, or terminate a run
- record both technical and business-level failure reasons
- prevent duplicate high-risk actions through idempotency keys or execution guards

19. Architecture Decisions

Key decisions in this architecture:

1. React + Django was selected over Django-only UI because the interactive flow builder and rich review workspaces benefit from a modern frontend.
2. LangChain is retained for agentic workflow orchestration, but bounded by policy, typed outputs, and backend mediation.
3. MCP is used as the execution plane for enterprise tools rather than embedding direct tool integrations throughout the application.
4. H2 is used initially, but the schema and persistence model are intentionally database-portable.
5. Governance, state management, approvals, and audit remain in the application layer rather than inside MCP or LangChain.

20. Future Architecture Extensions

Potential future enhancements:

- vector-based retrieval for code and document context
- richer simulation and dry-run environment for custom flows
- multi-tenant team isolation
- policy-as-code integration
- advanced analytics and optimization recommendations
- expanded environment promotion workflows beyond development
- support for additional agent frameworks in parallel with LangChain if needed

21. Appendix A: Reference Runtime Sequence

Example end-to-end sequence:

1. Developer selects Jira story in React UI.
2. Django creates workflow session.
3. LangChain Story Analysis Agent summarizes story using MCP story retrieval and relevant resources.

4. Developer approves plan.
5. Django invokes MCP branch creation tool.
6. Context Agent retrieves scoped repository context.
7. Code Agent generates proposed changes.
8. Developer reviews diffs and approves.
9. PR Agent drafts pull request through MCP.
10. Reviewer approves in review console.
11. Django invokes MCP deployment tool for development environment.
12. Audit, state, and metrics are updated throughout the process.

Appendix B: Simplified Architecture Views

```
View 1: Layered Architecture

[ React UI ]
      |
      v
[ Django REST API ]
      |
      v
[ Workflow Engine + Policy Layer ]
      |
      v
[ LangChain Orchestrator ]
      |
      v
[ MCP Client ]
      |
      v
[ MCP Server ]
      |
      +--> Jira
      +--> GitHub
      +--> CI/CD
      +--> Deploy
      +--> Repo Context

View 2: Persistence and Workers

[ Django API ] ----> [ H2 Database ]
      |
      +----> [ Worker Queue / Celery or RQ ]
                    |
                    +--> Agent Execution
                    +--> MCP Polling
                    +--> Deployment Status Checks
```